

“Specialization” of a dictionary may be appropriate when it is used to compress specialized data. However, we should caution the reader to note that all one can really claim about the statistics reported in Appendices A.1.1 through A.1.9 is that they reflect the files listed in Appendix A.1, give some loose indication about technical writing in computer science, and only a slight indication of English writing as a whole. In order to get accurate statistics about a source such as written English, even in a restricted environment such as computer science technical writing, much more data would have to be sampled (i.e., by at least several orders of magnitude over the sample size of 1 million characters used here).

1.4. COMPRESSION BY TEXTUAL SUBSTITUTION

A powerful (and practical) approach to compressing a string of characters is *textual substitution*. A textual substitution data compression method is any compression method that compresses text by identifying repeated substrings and replacing some substrings by references to other copies. We call such a reference a *pointer*[†] and the string to which the pointer refers the *target* of the pointer. Thus, the input to a data compression algorithm employing textual substitution is a sequence of characters over some alphabet Σ and the output is a sequence of characters from Σ interspersed with pointers. We make the distinction between pointers and characters only for convenience. Since any well defined compression algorithm must be able to take as input a string consisting of a single character, it must be possible to represent any single character with whatever coding scheme is used to represent the output from the algorithm. Hence, a character can be viewed as a pointer to a string of exactly one character and the output of the compression algorithm as simply a sequence of pointers, where each pointer specifies some string of ≥ 1 characters of Σ .

[†] In the literature, such references are often called *codewords* or *tokens*.

1.5. OFF-LINE VERSUS ON-LINE ALGORITHMS

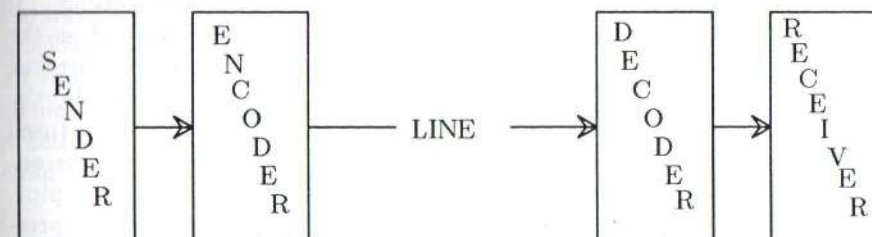


Figure 1.5.1

As depicted by Figure 1.5.1, a practical model for *on-line* data compression by textual substitution is to have an *encoder* and *decoder*, each with a fixed (finite) amount of local memory, which we refer to as a *local dictionary*, D . We assume that the two local dictionaries may be initialized at the beginning of time to contain identical information. The encoder connects the *communication line* to the *sender* and the decoder connects the communication line to the *receiver*. What distinguishes this model from an *off-line* model is that neither the sender nor the receiver can see all of the data at once; data must be constantly passing from the sender through the encoder, through the decoder, and on to the receiver. In theory, if D is large, then the encoder, for example, could read in as many bits of data as D is capable of storing (possibly more if the data is compressible), before having to send any bits to the receiver. However, in practice, the algorithms to be presented in this chapter can easily be implemented to run in real-time.

Definition 1.5.1: A compression-decompression method is *real-time* if there exists a constant k (which does not depend on the data being processed) such that for every k units of time, exactly one new character is read by the encoder and exactly one character is written by the decoder. The only exception to this rule is that we may allow a small “lag” between the encoder and decoder. That is, there is another constant l (that is independent of the data being processed) such that during the first l units of time the decoder may produce no characters and, in the case that the input is finite in length, l